NEXT ENGINEER · _SHEERAT_

Big Data

BOE099

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** ALL | **Semester:** 8 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 4th year, Open Elective-IV, 2025-26 onwards

UNIT 1	UNIT 2	UNIT 3	UNIT 4	UNIT 5
Introduction to Big Data	Hadoop and MapReduce	HDFS, Data Ingestion, Hadoop I/O and Cluster Setup	Hadoop Ecosystem, YARN, NoSQL, MongoDB, Spark and Scala	Pig, Hive, HBase, ZooKeeper and IBM BigInsights

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: architectures, components and comparison tables.
3. **Solved examples**: block and storage calculations, MapReduce data flow, and short code (Java MapReduce, MongoDB, Spark, Scala, Pig Latin, HiveQL, HBase shell).
4. **Code / key points to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (types of data, 5 Vs, architecture, analytics types, analysis vs reporting) and Unit 2 (Hadoop components, MapReduce flow, job run anatomy, shuffle and sort, input and output formats, word count).
- **Afternoon (3 hours):** Unit 3 (HDFS design, blocks, replication, read and write paths, Flume, Sqoop, Avro, cluster setup) and Unit 4 (schedulers, YARN, HDFS federation and HA, NoSQL, MongoDB CRUD, Spark RDD, Scala basics).
- **Evening (2 hours):** Unit 5 (Pig, Hive, HBase, ZooKeeper, BigInsights). Then the "Last-hour checklist".
- **In the exam:** draw the architecture (HDFS, YARN, Hive, HBase) and label every daemon. For MapReduce questions, show the key-value pairs after map, after shuffle and after reduce.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction to Big Data

✍️ 2-mark definitions

- **Big data:** data sets so large, fast-changing or varied that traditional databases and tools cannot store, process or analyse them in acceptable time.
- **Structured data:** data in a fixed schema of rows and columns (relational tables, spreadsheets); about 10% of enterprise data.
- **Semi-structured data:** data with tags or markers but no rigid schema (XML, JSON, email headers, log files).
- **Unstructured data:** data without a predefined model (text documents, images, audio, video, social media posts); the largest share (about 80%).
- **5 Vs of big data:** Volume (scale: terabytes to zettabytes), Velocity (speed of generation and processing: streams), Variety (many types and formats), Veracity (uncertainty and quality of data), Value (useful insight extracted). Some add Variability and Visualization.
- **Big data platform:** an integrated set of tools for storing, processing and analysing big data (for example Hadoop with Spark, Hive, HBase; or cloud services).
- **Big data analytics:** examining large data sets to find hidden patterns, correlations, trends and preferences that support decisions.
- **Intelligent data analysis (IDA):** using AI and statistical techniques (machine learning, data mining, pattern recognition) to extract knowledge from data.
- **Data lake:** a central repository storing raw data of all types in its native format until needed.
- **Data privacy:** the right of individuals to control how their personal data is collected and used.
- **Data governance and compliance:** policies and controls ensuring that data is accurate, secure and handled according to laws (GDPR, India's DPDP Act 2023, HIPAA).

💡 Core concepts

History of big data: 1990s data warehouses and OLAP; 2003 Google File System paper; 2004 Google MapReduce paper; 2006 Hadoop created by Doug Cutting and Mike Cafarella (named after a toy elephant) and adopted by Yahoo; 2008 Hadoop becomes a top-level Apache project; 2012 Hadoop 2 with YARN; 2014 Apache Spark becomes a top-level project; today cloud data platforms and real-time streaming.

Drivers of big data: cheap storage and commodity hardware, growth of the internet, social media, mobile devices and IoT sensors, cloud computing, open-source tools, and the business need for data-driven decisions.

Big data architecture (layers)

1. **Data sources:** databases, logs, sensors, social media, clickstreams.
2. **Ingestion layer:** batch (Sqoop) and streaming (Flume, Kafka).
3. **Storage layer:** HDFS, NoSQL (HBase, MongoDB, Cassandra), data lake, cloud object storage.
4. **Processing layer:** batch (MapReduce, Spark) and stream (Spark Streaming, Storm, Flink).

5. **Analysis and query layer:** Hive, Pig, Spark SQL, machine learning (Mahout, Spark MLlib).
6. **Visualization and consumption:** dashboards and reports (Tableau, Power BI).
7. **Cross-cutting:** security, governance, metadata, workflow (Oozie), coordination (ZooKeeper).

Technology components: distributed file system, distributed processing framework, NoSQL databases, in-memory computing, stream processing, MPP (massively parallel processing) databases, cloud, data integration tools.

Importance and applications: healthcare (disease prediction, patient records), banking (fraud detection, credit scoring), retail and e-commerce (recommendations, basket analysis), telecom (churn prediction, network optimization), manufacturing (predictive maintenance), government (smart cities, traffic management, Aadhaar), transport (route optimization), social media (sentiment analysis), agriculture (yield prediction).

Security, compliance, auditing and protection

- Security: authentication (Kerberos in Hadoop), authorization (Apache Ranger, Sentry; HDFS permissions and ACLs), encryption at rest and in transit, network security.
- Compliance: following laws and standards on storage, retention and processing of personal data.
- Auditing: logging who accessed or changed what data and when (Apache Ranger audit, Atlas lineage).
- Protection: backup, replication, disaster recovery, data masking and anonymization.

Big data privacy and ethics: consent and purpose limitation, anonymization (risk of re-identification by linking data sets), bias and fairness in algorithms, transparency, data ownership, surveillance concerns, and responsible use of personal data.

Types of analytics

Type	Question	Example
Descriptive	What happened?	Monthly sales report
Diagnostic	Why did it happen?	Drill-down showing sales fell in one region
Predictive	What will happen?	Forecast next quarter's demand
Prescriptive	What should we do?	Recommend optimal stock levels and prices

Challenges of conventional systems: RDBMS scale up (bigger servers) not out; fixed schemas cannot handle variety; high licence and hardware cost; poor performance on petabytes; not designed for unstructured or streaming data; single points of failure.

Nature of data and analytic process: data is collected, cleaned (missing values, duplicates, errors), integrated, transformed, analysed with statistics and machine learning, interpreted, and visualized. Analytic tools: Hadoop, Spark, R, Python (pandas, scikit-learn), SAS, Tableau, Power BI, KNIME, RapidMiner.

Analysis vs reporting

Reporting	Analysis
Organizes data into summaries (what happened)	Explores data to find why and what next
Pushes information to users (scheduled, standard)	Pulls insight in response to questions

Reporting	Analysis
Produces reports, dashboards, alerts	Produces findings and recommendations
Little interpretation	Needs interpretation and context

Modern data analytic tools: Apache Hadoop, Spark, Kafka, Flink, Hive, Presto/Trino, Databricks, Snowflake, Google BigQuery, Amazon EMR and Redshift, Azure Synapse, Elasticsearch, Tableau, Power BI.

🧮 Solved examples

Example 1 (classify data). Customer table in MySQL: structured. Product catalogue in JSON from an API: semi-structured. Customer call recordings and product review text: unstructured.

Example 2 (5 Vs for a ride-hailing app). Volume: billions of GPS points per day. Velocity: location updates every few seconds, real-time matching. Variety: GPS, payments, ratings, chat text, maps. Veracity: GPS noise, fake accounts. Value: dynamic pricing, shorter wait times, fraud detection.

Key points to remember

- 5 Vs: volume, velocity, variety, veracity, value.
- Structured ≈ 10%, unstructured ≈ 80% of data.
- Analytics: descriptive, diagnostic, predictive, prescriptive.
- RDBMS scales up; Hadoop scales out on commodity hardware.

⭐ Important questions

1. Define big data. Explain the types of digital data and the 5 Vs with examples.
2. Explain the history and drivers of big data and draw the big data architecture.
3. Explain applications of big data in different fields.
4. Discuss security, compliance, auditing, privacy and ethics in big data.
5. What are the challenges of conventional systems? Explain the analytic process and types of analytics. Differentiate analysis and reporting.

UNIT 2 Hadoop and MapReduce

✍️ 2-mark definitions

- **Apache Hadoop:** an open-source framework for distributed storage (HDFS) and processing (MapReduce, on YARN) of large data sets on clusters of commodity hardware.
- **Scaling out:** adding more machines to a cluster (horizontal scaling), as opposed to scaling up (a bigger machine).
- **Data locality:** moving computation to the node where the data block is stored rather than moving data across the network.
- **MapReduce:** a programming model in which a map function turns input records into intermediate key-value pairs and a reduce function aggregates all values for each key.
- **InputSplit:** the logical chunk of input processed by one map task (usually one HDFS block).
- **RecordReader:** converts an input split into key-value records for the mapper.
- **Combiner:** an optional "mini reducer" that runs on map output on the same node to reduce data sent over the network.
- **Partitioner:** decides which reducer receives each intermediate key (default: hash(key) mod number of reducers).
- **Shuffle and sort:** the phase that transfers map outputs to reducers, merges them and sorts by key.
- **Hadoop Streaming:** a utility that lets map and reduce functions be written in any language that reads standard input and writes standard output (Python, Ruby).
- **Hadoop Pipes:** a C++ interface to MapReduce using sockets.
- **MRUnit:** a Java library for unit-testing mappers and reducers without a cluster.
- **Speculative execution:** launching a duplicate copy of a slow task on another node and using whichever finishes first.

💡 Core concepts

Hadoop core components: Hadoop Common (shared utilities), HDFS (storage), YARN (resource management, Hadoop 2+), MapReduce (processing).

Hadoop ecosystem: Hive (SQL), Pig (data flow scripts), HBase (NoSQL column store), Sqoop (RDBMS import and export), Flume (log ingestion), Oozie (workflow scheduler), ZooKeeper (coordination), Mahout (machine learning), Spark (in-memory processing), Ambari (cluster management), Kafka (messaging).

Data formats: text (CSV, TSV, JSON lines), SequenceFile (binary key-value), Avro (row-based with schema), Parquet and ORC (columnar, compressed, good for analytics).

How MapReduce works (data flow)

1. Input files in HDFS are divided into **splits**.
2. Each split goes to a **map task**; the RecordReader gives (key, value) pairs; map emits intermediate (k2, v2) pairs.

3. Map output goes to a circular memory buffer (100 MB by default); when 80% full it is **partitioned**, **sorted** by key, optionally **combined**, and **spilled** to local disk; spills are merged into one sorted file.
4. **Shuffle:** reducers fetch their partitions from all mappers over HTTP.
5. **Sort/merge:** reducers merge the fetched data, grouping values by key.
6. **Reduce:** for each key with its list of values, reduce emits (k3, v3) pairs.
7. **Output:** written to HDFS via OutputFormat (one file per reducer: part-r-00000, ...).

Anatomy of a MapReduce job run (classic and YARN)

1. Client submits the job (`job.waitForCompletion()`): gets a job ID, checks output, computes splits, copies job JAR and config to HDFS.
2. The ResourceManager starts an **ApplicationMaster** (MRAppMaster) in a container.
3. The AM requests containers for map and reduce tasks, preferring data-local nodes.
4. NodeManagers launch tasks in **YarnChild** JVMs, which run map or reduce.
5. Tasks report progress and counters; the AM aggregates and reports to the client.
6. On completion, the AM and containers clean up; the job status becomes successful.

(In Hadoop 1 / MRv1 the JobTracker scheduled jobs and TaskTrackers ran tasks in fixed map and reduce slots.)

Failures

- **Task failure:** an exception or a hung task (no progress for 10 minutes); retried up to 4 times on other nodes.
- **ApplicationMaster failure:** restarted by the ResourceManager (2 attempts by default), recovering completed tasks.
- **NodeManager failure:** detected by missed heartbeats; tasks rescheduled elsewhere.
- **ResourceManager failure:** serious; handled by RM high availability with a standby and ZooKeeper.

Job scheduling: FIFO (default in early Hadoop), Fair Scheduler, Capacity Scheduler (see Unit 4).

MapReduce types and formats

- Types: map: (K1, V1) → list(K2, V2); combine: (K2, list(V2)) → list(K2, V2); reduce: (K2, list(V2)) → list(K3, V3).
- **Input formats:** TextInputFormat (default; key = byte offset, value = line), KeyValueTextInputFormat (split at a tab), NLineInputFormat (N lines per split), SequenceFileInputFormat, DBInputFormat, CombineFileInputFormat (many small files).
- **Output formats:** TextOutputFormat (default; key TAB value), SequenceFileOutputFormat, MultipleOutputs, LazyOutputFormat, DBOutputFormat.

MapReduce features: counters (built-in and user-defined), sorting (partial, total with TotalOrderPartitioner, secondary sort), joins (map-side join for sorted, partitioned inputs; reduce-side join using tagged records), side data distribution (distributed cache).

Real-world uses: web indexing, log analysis, ETL, recommendation pre-processing, large-scale sorting (TeraSort), click-stream analysis, genome sequencing.

🧮 Solved examples

Example 1 (word count data flow). Input lines: "big data is big" and "data is new".

NEXT ENGINEER _sheerat_

- Map output: (big,1) (data,1) (is,1) (big,1) | (data,1) (is,1) (new,1).
- Combiner on mapper 1: (big,2) (data,1) (is,1).
- After shuffle and sort: big → [2]; data → [1, 1]; is → [1, 1]; new → [1].
- Reduce output: **big 2, data 2, is 2, new 1**.

Example 2 (word count in Java).

```
public class WordCount {
  public static class TokenMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
    private final static IntWritable ONE = new IntWritable(1);
    private final Text word = new Text();
    public void map(LongWritable key, Text value, Context ctx)
        throws IOException, InterruptedException {
      for (String w : value.toString().toLowerCase().split("\\s+")) {
        if (w.isEmpty()) continue;
        word.set(w);
        ctx.write(word, ONE);
      }
    }
  }
  public static class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context ctx)
        throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable v : values) sum += v.get();
      ctx.write(key, new IntWritable(sum));
    }
  }
  public static void main(String[] args) throws Exception {
    Job job = Job.getInstance(new Configuration(), "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenMapper.class);
    job.setCombinerClass(SumReducer.class);
    job.setReducerClass(SumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Run: `hadoop jar wc.jar WordCount /input /output`.

Example 3 (Hadoop Streaming in Python).

```
# mapper.py
import sys
for line in sys.stdin:
    for w in line.split():
        print(f"{w.lower()}\t1")

# reducer.py (input arrives sorted by key)
import sys
cur, total = None, 0
for line in sys.stdin:
    w, c = line.rstrip("\n").split("\t")
    if w != cur:
        if cur is not None:
            print(f"{cur}\t{total}")
        cur, total = w, 0
    total += int(c)
if cur is not None:
    print(f"{cur}\t{total}")
```

`hadoop jar hadoop-streaming.jar -input /in -output /out -mapper mapper.py -reducer reducer.py -file mapper.py -file reducer.py`

Example 4 (MRUnit test).

```
MapDriver<LongWritable, Text, Text, IntWritable> md = MapDriver.newMapDriver(new
TokenMapper());
md.withInput(new LongWritable(0), new Text("big big"))
  .withOutput(new Text("big"), new IntWritable(1))
  .withOutput(new Text("big"), new IntWritable(1))
  .runTest();
```

Example 5 (maximum temperature). Map: parse each weather record → (year, temperature), skipping missing values. Reduce: (year, [temps]) → (year, max). Combiner = reducer works here because max is associative and commutative (it would not work directly for average).

Key points to remember

- Map: (K1, V1) → list(K2, V2); reduce: (K2, list(V2)) → list(K3, V3).
- Flow: split → map → buffer, partition, sort, combine, spill → shuffle → merge → reduce → output.
- Default partitioner: hash(key) mod R. Combiner only for associative and commutative operations.
- TextInputFormat key = byte offset.
- Task retried 4 times; speculative execution for slow tasks.

⭐ Important questions

1. Explain Apache Hadoop, its history and components. What is the Hadoop ecosystem?
2. Explain how MapReduce works with a diagram, using word count as an example.
3. Explain the anatomy of a MapReduce job run on YARN.

4. Explain failures in MapReduce and how they are handled. Explain job scheduling.
5. Explain shuffle and sort in detail. What are combiners and partitioners?
6. Explain MapReduce input and output formats. Write a MapReduce program and explain how to test it with MRUnit.
7. Explain Hadoop Streaming and Hadoop Pipes.

NEXT ENGINEER
sheerat

UNIT 3 HDFS, Data Ingestion, Hadoop I/O and Cluster Setup

✍️ 2-mark definitions

- **HDFS (Hadoop Distributed File System):** a distributed file system designed for very large files, streaming (write-once, read-many) access, on commodity hardware.
- **Block:** the unit of storage in HDFS; 128 MB by default in Hadoop 2+ (64 MB in Hadoop 1). A file smaller than a block does not occupy a full block on disk.
- **NameNode:** the master that stores the namespace (directory tree) and metadata (file → blocks, block → DataNodes) in memory; does not store data.
- **DataNode:** a worker that stores blocks on local disks and sends heartbeats and block reports to the NameNode.
- **Secondary NameNode:** periodically merges the edit log with the fsimage (checkpointing); it is not a hot standby.
- **fsimage and edit log:** fsimage is the snapshot of the namespace; the edit log records changes since the last snapshot.
- **Replication factor:** number of copies of each block (default 3).
- **Rack awareness:** placing replicas on different racks so that a rack failure does not lose data.
- **Apache Flume:** a distributed service to collect, aggregate and move large amounts of streaming log data into HDFS (source → channel → sink).
- **Apache Sqoop:** a tool to transfer bulk data between relational databases and Hadoop (import and export).
- **Hadoop archive (HAR):** packs many small files into a single archive to reduce NameNode memory use.
- **Serialization:** converting structured objects into a byte stream for storage or network transfer (Writable in Hadoop, Avro).
- **Apache Avro:** a language-neutral data serialization system with schemas written in JSON, compact binary format and schema evolution.

💡 Core concepts

Design of HDFS: very large files; streaming data access (high throughput rather than low latency); commodity hardware (failure is normal); write once, read many (appends allowed, no random updates).
Not suited for: low-latency access, many small files (each file and block uses about 150 bytes of NameNode memory), multiple writers or arbitrary modifications.

Benefits and challenges: benefits are fault tolerance through replication, scalability to thousands of nodes, high throughput, low cost, data locality. Challenges are the NameNode as a single point of failure (solved by HA in Hadoop 2), small-file problem, no random writes, and latency.

Why large blocks? To make seek time a small fraction of transfer time. With a seek time of 10 ms and transfer rate of 100 MB/s, a 100 MB block makes seek 1% of transfer time. The block abstraction also allows files larger than any one disk, simplifies storage management, and fits replication.

Replica placement (default policy): 1st replica on the writer's node (or a random node), 2nd on a node in a different rack, 3rd on a different node in the same rack as the 2nd. This balances reliability and write bandwidth.

HDFS read path

1. Client calls `open()` on DistributedFileSystem.
2. The NameNode returns the DataNode addresses for the first blocks, sorted by closeness.
3. The client reads from the closest DataNode through FSDataInputStream; at each block end it moves to the next DataNode.
4. Checksums are verified; on failure it tries another replica and reports the bad block.
5. `close()` the stream.

HDFS write path

1. Client calls `create()`; the NameNode checks permissions and that the file does not exist, then records it.
2. Data is split into packets and placed in a data queue.
3. The DataStreamer asks the NameNode for new blocks and a list of DataNodes (by replication factor).
4. Packets are sent along a **pipeline**: DataNode 1 → DataNode 2 → DataNode 3.
5. Acknowledgements return through the pipeline (ack queue); a packet is removed after all acknowledge.
6. On a DataNode failure, the pipeline is rebuilt with the remaining nodes, and the block is re-replicated later.
7. `close()` flushes packets and tells the NameNode the file is complete.

Interfaces: Java API (FileSystem, FSDataInputStream, FSDataOutputStream, Path), command line (`hdfs dfs` or `hadoop fs`), WebHDFS (REST over HTTP), HttpFS, libhdfs (C), NFS gateway, FUSE.

Common HDFS shell commands

```
hdfs dfs -mkdir -p /user/amit/input
hdfs dfs -put sales.csv /user/amit/input      # copy local to HDFS
hdfs dfs -ls /user/amit/input
hdfs dfs -cat /user/amit/input/sales.csv
hdfs dfs -get /user/amit/output/part-r-00000 . # copy HDFS to local
hdfs dfs -setrep 2 /user/amit/input/sales.csv
hdfs dfs -rm -r /user/amit/output
hdfs fsck /user/amit -files -blocks -locations
```

Java read example

```
FileSystem fs = FileSystem.get(URI.create(uri), new Configuration());
try (InputStream in = fs.open(new Path(uri))) {
    IOUtils.copyBytes(in, System.out, 4096, false);
}
```

Data ingestion

- **Flume:** agent = source (exec, spooling directory, HTTP, Kafka, syslog) → channel (memory or file) → sink (HDFS, HBase, Kafka, logger). Agents can be chained (fan-in, fan-out). Example: collect web server logs from many machines into HDFS directories by date.

- **Sqoop:** `sqoop import --connect jdbc:mysql://db/shop --username u -P --table orders --target-dir /data/orders -m 4` runs a map-only job with 4 mappers splitting on the primary key. `sqoop export` writes HDFS data back to a table. Options for Hive import, incremental import (append or lastmodified), and free-form queries.

Hadoop I/O

- **Data integrity:** CRC-32C checksums computed per 512 bytes; verified on read; DataNodes run a block scanner.
- **Compression:** codecs gzip (DEFLATE; not splittable), bzip2 (splittable, slow), LZO (fast, splittable with an index), Snappy (very fast, not splittable alone), LZ4, Zstandard. Use splittable formats or container formats (SequenceFile, Avro, Parquet, ORC) for MapReduce inputs. Compressing map output reduces shuffle traffic.
- **Serialization:** Writable interface (`write(DataOutput)` , `readFields(DataInput)`) with types IntWritable, LongWritable, Text, NullWritable, ArrayWritable, MapWritable. WritableComparable for keys (must support sorting). Compact, fast and interoperable.
- **Avro:** schema in JSON, data in binary; schema stored with the data file; supports schema evolution (add fields with defaults); works with many languages; Avro data files are splittable and compressible.
- **File-based data structures:** SequenceFile (binary key-value with sync markers; uncompressed, record- or block-compressed), MapFile (sorted SequenceFile with an index for lookups).

Hadoop cluster setup

- **Cluster specification:** commodity servers (multi-core CPUs, 64 to 512 GB RAM, several large disks in JBOD, not RAID), 10 Gb Ethernet, racks of 20 to 40 nodes with a top-of-rack switch. Master nodes (NameNode, ResourceManager) need more memory and reliable hardware.
- **Installation:** install Java, create a hadoop user, set up passwordless SSH, download and unpack Hadoop, set JAVA_HOME and HADOOP_HOME, format the NameNode (`hdfs namenode -format`), start daemons (`start-dfs.sh` , `start-yarn.sh`).
- **Configuration files:** core-site.xml (`fs.defaultFS`), hdfs-site.xml (`dfs.replication` , `dfs.blocksize` , NameNode and DataNode directories), yarn-site.xml (ResourceManager address, NodeManager memory), mapred-site.xml (`mapreduce.framework.name = yarn`), workers (list of worker hosts), hadoop-env.sh.
- **Security:** Kerberos authentication, service-level authorization, HDFS permissions and ACLs, encryption zones, Ranger and Knox.
- **Administration and maintenance:** commissioning and decommissioning nodes, balancer (`hdfs balancer`), safe mode, fsck, backups of NameNode metadata, upgrades.
- **Monitoring:** web UIs (NameNode on port 9870, ResourceManager on 8088), JMX metrics, logs, Ambari, Ganglia, Nagios.
- **Benchmarks:** TeraGen/TeraSort/TeraValidate (sorting), TestDFSIO (HDFS I/O throughput), NNBench (NameNode load), MRBench (small jobs).
- **Hadoop in the cloud:** Amazon EMR, Azure HDInsight, Google Dataproc; storage in S3, ADLS or GCS; clusters started on demand and shut down after jobs.

🧮 Solved examples

Example 1 (blocks and storage). A 1 GB (1,024 MB) file with 128 MB blocks and replication 3.

- Blocks = 1024/128 = **8 blocks**; block replicas = 8 × 3 = **24**; raw storage used = 1 GB × 3 = **3 GB**.
- A 500 MB file → 3 full blocks (384 MB) + 1 block of 116 MB = **4 blocks**; storage = 1,500 MB (the last block uses only 116 MB on disk, not 128).

Example 2 (map tasks). 10 files of 300 MB each with 128 MB blocks: each file has 3 blocks (128 + 128 + 44), so **30 splits = 30 map tasks**. If the same 3 GB were stored as 30,000 small files of 100 KB, there would be 30,000 map tasks and 30,000 NameNode entries: the small-file problem. Fix: HAR, SequenceFile or CombineFileInputFormat.

Example 3 (NameNode memory). 10 million files, each with 1 block: about 2 objects per file (file + block) × 150 bytes ≈ **3 GB** of NameNode heap.

Example 4 (Avro schema).

```
{"type": "record", "name": "Student",
 "fields": [{"name": "roll", "type": "int"},
            {"name": "name", "type": "string"},
            {"name": "cgpa", "type": ["null", "float"], "default": null}]}
```

Key points to remember

- Block 128 MB, replication 3, rack-aware placement (1 local, 2 on one remote rack).
- NameNode: metadata in memory; DataNode: blocks; Secondary NameNode: checkpoint only.
- Read: NameNode for locations, then read directly from DataNodes. Write: pipeline of DataNodes with acks.
- Flume: source → channel → sink. Sqoop: RDBMS ↔ HDFS via map-only jobs.
- gzip not splittable; bzip2 splittable; Snappy fast.
- Config files: core-site, hdfs-site, yarn-site, mapred-site.

⭐ Important questions

1. Explain the design of HDFS, its concepts, benefits and challenges. Why are HDFS blocks large?
2. Explain block replication and rack awareness.
3. Explain how files are read and written in HDFS with diagrams.
4. Explain Java interfaces and command-line interface of HDFS.
5. Explain Flume and Sqoop with examples. What are Hadoop archives?
6. Explain Hadoop I/O: data integrity, compression, serialization, Avro, SequenceFile and MapFile.
7. Explain Hadoop cluster setup: specification, installation, configuration, security, administration, monitoring and benchmarks. How is Hadoop run in the cloud?

UNIT 4 Hadoop Ecosystem, YARN, NoSQL, MongoDB, Spark and Scala

✍️ 2-mark definitions

- **YARN (Yet Another Resource Negotiator):** the resource management layer of Hadoop 2+, separating resource management from data processing, so that many engines (MapReduce, Spark, Tez) can share a cluster.
- **ResourceManager (RM):** the master daemon that allocates cluster resources (scheduler + applications manager).
- **NodeManager (NM):** the per-node agent that launches and monitors containers and reports resource use.
- **ApplicationMaster (AM):** a per-application process that negotiates containers from the RM and works with NMs to run tasks.
- **Container:** a bundle of resources (memory, CPU cores) on a node, in which a task runs.
- **Fair Scheduler:** gives every running application (or pool) an equal share of resources over time.
- **Capacity Scheduler:** divides the cluster into queues with guaranteed capacities for different organizations; unused capacity can be borrowed.
- **NameNode High Availability:** an active and a standby NameNode share the edit log (through JournalNodes); ZooKeeper-based failover controllers switch automatically.
- **HDFS federation:** multiple independent NameNodes, each managing a part of the namespace (a namespace volume), sharing the same DataNodes; improves scalability.
- **NoSQL:** non-relational databases designed for large scale, flexible schemas and horizontal scaling: key-value, document, column-family and graph stores.
- **CAP theorem:** a distributed system can guarantee at most two of Consistency, Availability and Partition tolerance at the same time.
- **MongoDB:** an open-source document database that stores data as BSON (binary JSON) documents in collections.
- **Capped collection:** a fixed-size MongoDB collection that keeps insertion order and overwrites the oldest documents when full (useful for logs).
- **Apache Spark:** a fast, general-purpose cluster computing engine with in-memory processing and APIs for Scala, Java, Python and R.
- **RDD (Resilient Distributed Dataset):** Spark's basic abstraction: an immutable, partitioned collection of records that can be rebuilt from its lineage if lost.
- **Scala:** a statically typed language running on the JVM that combines object-oriented and functional programming; Spark is written in Scala.

💡 Core concepts

Hadoop 1 vs Hadoop 2

Hadoop 1 (MRv1)	Hadoop 2 (YARN)
JobTracker does resource management and job scheduling	RM manages resources; per-job ApplicationMaster manages tasks
TaskTrackers with fixed map and reduce slots	NodeManagers with flexible containers
Only MapReduce	MapReduce, Spark, Tez, Storm and others
Scales to about 4,000 nodes	Scales to 10,000+ nodes
NameNode single point of failure	NameNode HA and federation

Running an application on YARN: client submits to RM → RM finds a container to start the AM → AM registers with RM and requests containers → RM allocates containers on NMs → AM asks NMs to launch tasks → tasks report to AM → AM unregisters when done. **MRv1 programs on YARN:** existing MapReduce jobs run unchanged on YARN through MRAppMaster (binary compatibility for the `mapred` API; source compatibility for `mapreduce`).

Schedulers compared

FIFO	Capacity	Fair
One queue, jobs in order of submission	Multiple queues with guaranteed capacity per organization; FIFO within a queue by default	Resources shared equally among running jobs or pools over time
Small jobs wait behind large ones	Elastic: idle capacity can be used by other queues	Small jobs finish quickly; supports pre-emption and weights
Not suitable for shared clusters	Default in Apache Hadoop	Developed by Facebook

NoSQL types

Type	Data model	Examples	Use cases
Key-value	Key → value (opaque)	Redis, DynamoDB, Riak	Caching, session store, shopping carts
Document	Key → JSON/BSON document	MongoDB, CouchDB	Content management, catalogues, user profiles
Column-family	Row key → column families → columns	HBase, Cassandra	Time series, IoT, very large sparse tables
Graph	Nodes and edges with properties	Neo4j, JanusGraph	Social networks, recommendations, fraud rings

RDBMS vs NoSQL: RDBMS has fixed schema, ACID transactions, SQL, joins and vertical scaling; NoSQL has flexible schema, BASE (basically available, soft state, eventual consistency), horizontal scaling (sharding), and is optimized for specific access patterns.

MongoDB

- Terminology: database → collection (like a table) → document (like a row) → field (like a column). Every document has a unique `_id` (ObjectId by default).

- **Data types:** String, Integer (32 and 64-bit), Double, Decimal128, Boolean, Date, ObjectId, Array, embedded Document, Null, Binary data, Timestamp, Regular expression.
- **Indexing:** single-field, compound, multikey (arrays), text, geospatial (2dsphere), hashed (for sharding), TTL indexes; `explain()` shows whether an index is used.
- **Replication:** replica sets (one primary, several secondaries, automatic failover). **Sharding:** horizontal partitioning by shard key across shards, with mongos routers and config servers.
- **Aggregation pipeline:** $match, $group, $sort, $project, $lookup (join), $unwind, $limit.

Apache Spark

- **Why Spark:** keeps intermediate data in memory (up to 100× faster than MapReduce for iterative algorithms), rich API, interactive shell, unified engine.
- **Components:** Spark Core (RDDs, scheduling), Spark SQL (DataFrames), Spark Streaming / Structured Streaming, MLlib (machine learning), GraphX (graphs).
- **Installation:** install Java (and Scala), download Spark, set SPARK_HOME and PATH, run `spark-shell` (Scala) or `pyspark`; submit applications with `spark-submit`.
- **RDD operations:** transformations (lazy, return a new RDD: map, filter, flatMap, union, distinct, groupByKey, reduceByKey, join) and actions (trigger execution, return a value: collect, count, first, take, reduce, saveAsTextFile). Narrow dependencies (map, filter) need no shuffle; wide dependencies (reduceByKey, join) need a shuffle.
- **Applications, jobs, stages and tasks:** an application (one SparkContext) runs jobs; each action creates a **job**; the DAG scheduler splits a job into **stages** at shuffle boundaries; each stage has one **task** per partition, run by executors.
- **Anatomy of a Spark job run:** driver program creates SparkContext → connects to the cluster manager (standalone, YARN, Kubernetes) → executors are started on worker nodes → action triggers a job → DAG scheduler builds stages → task scheduler sends tasks to executors → results returned to the driver.
- **Spark on YARN:** client mode (driver runs on the client machine; good for interactive work) and cluster mode (driver runs inside the YARN ApplicationMaster; good for production). Executors run in YARN containers.
- **Fault tolerance:** lost partitions are recomputed from lineage; `persist()` / `cache()` keeps RDDs in memory or on disk.

Scala essentials

- **Classes and objects:** `class` defines a blueprint; `object` defines a singleton (used for `main` and companion objects); `case class` gives immutable data classes with built-in equality and pattern matching.
- **Basic types and operators:** Byte, Short, Int, Long, Float, Double, Char, String, Boolean, Unit; operators are methods (`1 + 2` is `1.+(2)`); `val` (immutable) vs `var` (mutable); type inference.
- **Control structures:** `if` is an expression that returns a value; `while`; `for` comprehensions with generators, filters and `yield`; `match` (pattern matching); `try/catch/finally`.
- **Functions and closures:** functions are first-class values; anonymous functions `(x: Int) => x * 2`; higher-order functions (map, filter, reduce); a closure is a function that captures variables from its surrounding scope.
- **Inheritance:** `extends` for single class inheritance; `override` keyword required; abstract classes; **traits** (like interfaces with implementation) mixed in with `with`.

🧮 Solved examples

Example 1 (MongoDB CRUD and queries).

```
use college
db.students.insertOne({ roll: 101, name: "Asha", branch: "CSE", cgpa: 8.6, skills: ["java",
"sql"] })
db.students.insertMany([
  { roll: 102, name: "Ravi", branch: "IT",  cgpa: 7.9 },
  { roll: 103, name: "Neha", branch: "CSE", cgpa: 9.1 }
])
db.students.find({ branch: "CSE", cgpa: { $gt: 8.5 } }, { name: 1, cgpa: 1, _id: 0 })
db.students.find().sort({ cgpa: -1 }).limit(2)
db.students.updateOne({ roll: 102 }, { $set: { cgpa: 8.2 }, $push: { skills: "python" } })
db.students.deleteOne({ roll: 103 })
db.students.createIndex({ branch: 1, cgpa: -1 })
db.students.aggregate([
  { $group: { _id: "$branch", avgCgpa: { $avg: "$cgpa" }, count: { $sum: 1 } } }
])
db.createCollection("logs", { capped: true, size: 1048576, max: 1000 })
```

Example 2 (Spark word count in Scala).

```
val lines  = sc.textFile("hdfs:///data/input.txt")
val counts = lines.flatMap(_.split("\\s+"))
                  .filter(_.nonEmpty)
                  .map(w => (w.toLowerCase, 1))
                  .reduceByKey(_ + _)          // wide dependency: shuffle, new stage
counts.saveAsTextFile("hdfs:///data/wc_out")  // action: triggers the job
```

The action creates one job with **2 stages**: stage 1 runs textFile, flatMap, filter and map (narrow dependencies, pipelined together); the shuffle for reduceByKey starts stage 2. With 4 input partitions, stage 1 runs 4 tasks.

Example 3 (Scala basics).

```
abstract class Shape { def area: Double }
class Circle(r: Double) extends Shape { override def area: Double = math.Pi * r * r }
trait Named { def label: String = getClass.getSimpleName }       // trait with code
case class Rect(w: Double, h: Double) extends Shape with Named { def area: Double = w * h }

object Demo {
  def main(args: Array[String]): Unit = {
    val shapes = List(new Circle(1.0), Rect(2, 3))
    val total  = shapes.map(_.area).sum                          // higher-order function
    val factor = 2
    val scale  = (x: Double) => x * factor                       // closure captures factor
    for (s <- shapes if s.area > 3) println(s.area)              // for with filter
    val kind = shapes.head match { case Rect(_, _) => "rect"; case _ => "other" }
    println(f"$total%.2f ${scale(total)}%.2f $kind ${Rect(1, 1).label}") // 9.14 18.28 other
Rect
  }
}
```

Key points to remember

- YARN: RM (scheduler + applications manager), NM, AM, containers.
- Schedulers: FIFO, Capacity (queues), Fair (equal share).
- NameNode HA: active + standby + JournalNodes + ZKFC. Federation: many NameNodes, shared DataNodes.
- NoSQL: key-value, document, column, graph; CAP; BASE.
- MongoDB: insertOne/find/updateOne/deleteOne, $gt, $set, $push, createIndex, aggregate, capped collections.
- Spark: transformations lazy, actions trigger; stages split at shuffles; tasks = partitions.
- Scala: val/var, case class, trait, closures, pattern matching.

⭐ Important questions

1. Explain the Fair and Capacity schedulers.
2. Explain Hadoop 2.0 features: NameNode high availability, HDFS federation, MRv2, YARN, and running MRv1 programs on YARN.
3. Explain the YARN architecture and how an application runs on it.
4. What is NoSQL? Explain its types and compare it with RDBMS.
5. Explain MongoDB data types, CRUD operations, querying, indexing and capped collections.
6. Explain Spark: installation, RDDs, applications, jobs, stages and tasks, anatomy of a job run, and Spark on YARN.
7. Explain Scala classes and objects, types and operators, control structures, functions and closures, and inheritance.

UNIT 5 Pig, Hive, HBase, ZooKeeper and IBM BigInsights

✍️ 2-mark definitions

- **Apache Pig:** a high-level platform for analysing large data sets, using the data-flow language **Pig Latin**, which is compiled into MapReduce (or Tez/Spark) jobs.
- **Grunt:** Pig's interactive shell.
- **Pig execution modes:** local mode (`pig -x local` , local file system, for testing) and MapReduce mode (default; runs on a Hadoop cluster with HDFS).
- **Pig data model:** atom (simple value), tuple (ordered set of fields), bag (collection of tuples), map (key-value pairs); a relation is an outer bag.
- **UDF (User Defined Function):** a custom function written in Java, Python or other languages and used inside Pig or Hive.
- **Apache Hive:** a data warehouse system on Hadoop that lets users query data with HiveQL (SQL-like), compiled into MapReduce, Tez or Spark jobs.
- **Metastore:** Hive's central repository of metadata (table schemas, partitions, locations), usually stored in an RDBMS (Derby, MySQL).
- **Managed vs external table:** dropping a managed (internal) table deletes both metadata and data; dropping an external table deletes only metadata.
- **Partitioning and bucketing (Hive):** partitioning splits a table into directories by column value (for example by year); bucketing splits data into a fixed number of files by hash of a column.
- **Apache HBase:** a distributed, column-oriented NoSQL database on HDFS, modelled on Google's Bigtable, for random, real-time read/write access to huge tables.
- **Region:** a contiguous range of rows of an HBase table, served by a RegionServer.
- **Apache ZooKeeper:** a centralized coordination service for distributed systems, providing configuration management, naming, synchronization, leader election and group membership.
- **znode:** a data node in ZooKeeper's hierarchical namespace (persistent, ephemeral or sequential).
- **IBM InfoSphere BigInsights:** IBM's enterprise distribution of Hadoop, adding analytic tools such as BigSheets (spreadsheet-style analysis) and Big SQL (ANSI SQL on Hadoop).

💡 Core concepts

Pig vs databases (and MapReduce)

Pig	RDBMS / SQL	Java MapReduce
Procedural data flow (step by step)	Declarative queries	Low-level Java code
Schema optional, applied at load time	Fixed schema defined before loading	No schema
Works on files in HDFS, batch	Tables, transactions, random access, indexes	Files in HDFS

Pig	RDBMS / SQL	Java MapReduce
Nested data types (bags, tuples, maps)	Flat tables	Any Writable
About 10× fewer lines than MapReduce		Most code, most control

Pig Latin operators: LOAD, STORE, DUMP (display), FILTER, FOREACH ... GENERATE, GROUP, COGROUP, JOIN, ORDER BY, DISTINCT, LIMIT, UNION, SPLIT, FLATTEN, CROSS, SAMPLE; diagnostic operators DESCRIBE, EXPLAIN, ILLUSTRATE. Built-in functions: COUNT, SUM, AVG, MAX, MIN, CONCAT, TOKENIZE, SIZE.

Hive architecture

- **Clients/interfaces:** Hive CLI and Beeline shell, JDBC/ODBC drivers, web UI (HWI, Hue).
- **Services:** HiveServer2 (accepts queries from remote clients), Driver (manages the query life cycle), Compiler (parses HiveQL, checks the metastore, makes a logical plan), Optimizer, Execution engine (MapReduce, Tez or Spark).
- **Metastore:** embedded (Derby, single user), local or remote (MySQL/PostgreSQL shared by many users).
- **Storage:** HDFS (warehouse directory /user/hive/warehouse) or HBase.
- **Hive vs traditional database:** schema on read (checked when querying) vs schema on write; batch, high latency vs low latency; limited updates (ACID tables available) vs full transactions; scales to petabytes vs smaller; HiveQL vs full SQL.

HiveQL features: CREATE TABLE with ROW FORMAT and STORED AS (TEXTFILE, ORC, PARQUET, AVRO), LOAD DATA, INSERT ... SELECT, partitions and buckets, SELECT with WHERE, GROUP BY, HAVING, ORDER BY (total order, one reducer), SORT BY (per reducer), DISTRIBUTE BY, CLUSTER BY; joins (inner, left/right/full outer, left semi, map-side join), subqueries, views, built-in functions and UDFs (UDF, UDAF, UDTF), and calling external MapReduce scripts with TRANSFORM ... USING.

HBase

- **Data model:** table → rows (sorted by row key) → column families (defined at table creation) → columns (qualifiers, added freely) → cells with versions (timestamps). Sparse: empty cells take no space.
- **Architecture:** HMaster (assigns regions, handles schema changes, load balancing), RegionServers (serve regions; each has a write-ahead log (WAL), MemStore per column family, and HFiles in HDFS), ZooKeeper (tracks live servers and the location of the meta table).
- **Write path:** write to WAL → MemStore → flush to HFile when full; compactions merge HFiles (minor and major).
- **Clients:** HBase shell, Java API (Put, Get, Scan, Delete), REST (Stargate), Thrift, MapReduce integration, Phoenix (SQL layer).
- **HBase vs RDBMS:** column-oriented and schema-flexible vs row-oriented and fixed schema; scales out to billions of rows vs scale up; no joins or SQL natively vs SQL with joins; single-row atomicity vs multi-row ACID.
- **Schema design:** design the row key around access patterns (rows are sorted by key, so related rows should be adjacent); avoid hot-spotting from sequential keys (use salting, hashing or reversed

NEXT ENGINEER

timestamps); keep few column families (1 to 3); keep names short; use tall-narrow tables for time-series.

- **Advanced indexing:** HBase indexes only the row key; secondary indexes are built with separate index tables, coprocessors, Phoenix secondary indexes or external search (Solr, Elasticsearch).

ZooKeeper: an ensemble of servers (odd number, for majority quorum) with one leader; clients read from any server, writes go through the leader (ZAB protocol). Guarantees: sequential consistency, atomicity, single system image, reliability, timeliness. Uses: Hadoop and HBase master election, NameNode failover, Kafka broker coordination, configuration storage, distributed locks.

IBM InfoSphere BigInsights

- Enterprise Hadoop distribution with installation tools, a web console, security and management.
- **BigSheets:** a browser-based, spreadsheet-like tool for business users to collect, filter, combine and visualize big data without programming (built on Pig-like processing).
- **Big SQL:** an ANSI SQL engine on Hadoop that queries Hive tables and HDFS data with standard SQL, JDBC/ODBC and low latency through an MPP engine.
- Other features: text analytics (AQL), machine learning, integration with IBM databases and tools.

🧮 Solved examples

Example 1 (Pig Latin: top branches by average marks).

```
students = LOAD '/data/students.csv' USING PigStorage(',')
           AS (roll:int, name:chararray, branch:chararray, marks:int);
passed   = FILTER students BY marks >= 40;
grp      = GROUP passed BY branch;
avgm     = FOREACH grp GENERATE group AS branch, AVG(passed.marks) AS avg_marks,
                                COUNT(passed) AS n;
sorted   = ORDER avgm BY avg_marks DESC;
top3     = LIMIT sorted 3;
DUMP top3;
STORE sorted INTO '/out/branch_avg' USING PigStorage(',');
```

Example 2 (Pig word count).

```
lines = LOAD '/data/text' AS (line:chararray);
words = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) AS word;
g     = GROUP words BY word;
wc    = FOREACH g GENERATE group, COUNT(words);
DUMP wc;
```

Example 3 (HiveQL).

```
CREATE EXTERNAL TABLE sales (
  order_id INT, cust_id INT, amount DOUBLE, city STRING)
PARTITIONED BY (year INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/data/sales';

LOAD DATA INPATH '/staging/sales_2025.csv' INTO TABLE sales PARTITION (year = 2025);

SELECT city, SUM(amount) AS revenue
FROM sales
WHERE year = 2025
GROUP BY city
HAVING SUM(amount) > 100000
ORDER BY revenue DESC
LIMIT 5;

SELECT c.name, s.amount
FROM sales s JOIN customers c ON (s.cust_id = c.id)
WHERE s.year = 2025;

CREATE TABLE sales_orc STORED AS ORC AS SELECT * FROM sales;
```

The WHERE year = 2025 clause reads only that partition's directory (partition pruning).

Example 4 (HBase shell).

```
create 'student', 'info', 'marks'
put 'student', '101', 'info:name', 'Asha'
put 'student', '101', 'marks:dbms', '86'
get 'student', '101'
scan 'student', {COLUMNS => ['info:name'], LIMIT => 10}
delete 'student', '101', 'marks:dbms'
disable 'student'
drop 'student'
```

Key points to remember

- Pig: local and MapReduce modes; Grunt; LOAD, FILTER, FOREACH GENERATE, GROUP, JOIN, ORDER, STORE, DUMP; atom, tuple, bag, map.
- Hive: metastore, HiveServer2, driver, compiler, execution engine; schema on read; managed vs external; partition vs bucket; ORDER BY vs SORT BY.
- HBase: row key, column family, qualifier, timestamp; HMaster, RegionServer, WAL, MemStore, HFile; row-key design.
- ZooKeeper: znodes, ensemble, leader, quorum.
- BigInsights: BigSheets (spreadsheet UI), Big SQL (ANSI SQL).

⭐ Important questions

1. Explain Pig: execution modes, Grunt, Pig Latin, operators and UDFs. Compare Pig with databases and MapReduce. Write a Pig script.
2. Explain the Hive architecture, shell, services and metastore. Compare Hive with traditional databases.
3. Explain HiveQL: tables, querying, sorting and aggregation, joins, subqueries, UDFs and MapReduce scripts.
4. Explain the HBase data model and architecture. Compare HBase with RDBMS and explain schema design and indexing.
5. What is ZooKeeper? Explain its architecture and uses.
6. Write notes on IBM InfoSphere BigInsights, BigSheets and Big SQL.

Last-hour checklist

- ☐ Types of data; 5 Vs; history (GFS 2003, MapReduce 2004, Hadoop 2006); drivers.
- ☐ Big data architecture layers; applications; security, privacy and ethics.
- ☐ Four types of analytics; analysis vs reporting; conventional system challenges.
- ☐ Hadoop components and ecosystem; MapReduce flow; word count pairs.
- ☐ Job run on YARN; failures; shuffle and sort; combiner and partitioner; input and output formats; MRUnit; streaming.
- ☐ HDFS design; block 128 MB; replication 3; rack awareness; read and write paths.
- ☐ Block calculations (1 GB → 8 blocks, 3 GB stored); small-file problem.
- ☐ Flume, Sqoop, HAR; compression codecs; Writable; Avro; SequenceFile.
- ☐ Cluster setup files and benchmarks; Hadoop in the cloud.
- ☐ Schedulers; Hadoop 1 vs 2; YARN components; HA and federation.
- ☐ NoSQL types; CAP; MongoDB commands; capped collection.
- ☐ Spark components; RDD transformations vs actions; jobs, stages, tasks; Spark on YARN.
- ☐ Scala classes, case classes, traits, closures, for and match.
- ☐ Pig operators and script; Hive architecture and HiveQL; HBase model, architecture and shell; ZooKeeper; BigInsights.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Big Data, Big Analytics: Emerging Business Intelligence and Analytic Trends	Michael Minelli, Michele Chambers, Ambiga Dhiraj	Wiley	WorldCat
2	Big Data Black Book	DT Editorial Services	Wiley / Dreamtech	WorldCat
3	Understanding Big Data: Analytics for Enterprise Class Hadoop and Streaming Data	Dirk deRoos, Chris Eaton, George Lapis, Paul Zikopoulos, Tom Deutsch	McGraw Hill	WorldCat
4	Big Data Fundamentals: Concepts, Drivers and Techniques	Thomas Erl, Wajid Khattak, Paul Buhler	Prentice Hall	WorldCat

Syllabus source: AKTU Open Elective-III and IV syllabus for B.Tech 8th semester, 2025-26 (copy hosted by an affiliated college, PDF)

NEXT ENGINEER _sheerat_